MSR Data Challenge: GitSkills Checkpoint 2 Peer Review Response

Morgan Nicholson, Samuel Bodenheimer, and Aidan Sullivan
School of Informatics, Computing and Cyber Systems
CS599: Open Source Software, Dr. Igor Steinmacher

November 3, 2026

1 Response to peer review

Another group read the checkpoint report and the pilot. Their notes are the first review[1] and the second review[2], and our action points[3] record a decision on each point about the report, the scripts, or the paper. Section 3 shows each revised passage, and the full diff is in pull request 45.[4]

What we changed

- Future Work now says we will search published sources for a freedom category mapping and fix that mapping before any license is placed in a group.[5]
- We rewrote Method in shorter sentences. The population rules, the cutoff, and the quality gate are unchanged.[6]

[1] `https://github.com/nichmorgan/msr-data-challenge-2027/blob/main/docs/checkpoint_2/feedbacks/raw/f1.md`
[2] `https://github.com/nichmorgan/msr-data-challenge-2027/blob/main/docs/checkpoint_2/feedbacks/raw/f2.md`
[3] Section 4
[4] `https://github.com/nichmorgan/msr-data-challenge-2027/pull/45`
[5] AP-11 Write the freedom mapping before seeing the data
[6] AP-07 Shorter method sentences

2 Authors' reflection

We spent two weeks making a model extract taught tools from skill files, and before the peer review we had already started to discuss dropping that approach and labeling the tools by hand. That path needs the steps below.

- Find a sample size calculator, backed by a published source, that sets how many skills we need to label.
- Start with a small part of that sample, then return to review the labeling strategy.
- Find a labeling tool that keeps the labels in sync between members.
- Define the labels for tools and licenses, backed by a published source.

Because this discussion is still underway, we are not making the change in Checkpoint 2 or in this version of the paper.

3 Change report

This section shows each passage of the paper that changed after the review, first as it read and then as it reads now, with citations and section numbers as they appear in the paper.

Method (Section IV)[7]

Opening paragraph

Before

> We analyze the local copy of GitSkills with Python and Polars DataFrames. Every notebook sets one shared random seed when it loads the tables, and each sampled draw we report uses a fixed seed, so a rerun returns the same rows.

After

> We analyze the local copy of GitSkills with Python and Polars DataFrames. Each notebook sets one shared random seed when it loads the tables, and every sampled draw we report uses a fixed seed, so a rerun returns the same rows.

Taught-tool extraction (Section IV-B)

Before

> Extraction reads only the skill body and never the companion text, and it skips a body that is not plain text. We extract names with GLiNER [5], which gives each hit a score, and a cutoff on that score decides which hits we keep. For taught-tool extraction, we set the score cutoff at 0.8 and the target accuracy at 80%, and both values are arbitrary choices for this paper. The manual validation in Section IV-C checks that cutoff.
>
> The label-only run in Section V shows that labels alone accept false positives, such as the skill filename or GitHub inside an address. Improvements to this step are proposed as future work in Section VII-A.

[7] AP-07 Shorter method sentences

After

Extraction reads only the skill body, never the companion text. We extract names with GLiNER [5], which scores each hit, and a cutoff of 0.8 on that score decides which hits we keep. The cutoff and a target accuracy of 80% are both arbitrary choices for this paper, and the manual validation in Section IV-C checks the cutoff against that target. In the label-only run in Section V, labels alone accepted false positives, and Section VII-A proposes improvements to this step.

Manual validation (Section IV-C)

Before

A human check will act as a quality gate for every step that relies on a model, which are taught-tool extraction, the choice of an owning repository when a tool is not already linked, and license gaps when a model helps fill them.

We will label the taught tools in a random sample of the population, drawn with the shared seed and sized before anyone sees the model output. From those labels we will report precision, the share of extracted tools that are truly taught tools, and recall, the share of truly taught tools the model finds, along with inter-rater agreement on a subset that two authors label independently. If a stage falls short, we will change its labels or cutoff and run it again. Once it meets the target accuracy set in Section IV-B, we will fill the remaining gaps manually, merge the automatic and manual results while dropping entities that do not exist, and only then move on. Repository resolution and model-assisted license steps, described in Section VII, will use the same gate on their own samples.

Without model help, the same steps would mean full manual extraction, and checking every model output manually would take about as much effort. Either path would limit how many skills, tools, and repositories we can cover. We will therefore accept a bounded residual error under the sample gate, in the same spirit as a survey that estimates from a sample instead of visiting every case, so the study can scale. The gate is meant to catch the failures we expect from these models. In extraction, those include treating the folder that hosts the skill, the repository that publishes it, an attribution link, the skill filename, or GitHub inside an address as a taught tool.

After

Every step that relies on a model will go through a human quality gate. Those steps are taught-tool extraction, the choice of an owning repository when a tool is not already linked, and license gaps when a model helps fill them.

We will label the taught tools in a seeded random sample of the population, sized before anyone sees the model output, and report precision and recall against those labels. Precision is the share of extracted tools that are truly taught tools, and recall is the share of truly taught tools the model finds. Two authors will label a subset independently, and we will report their agreement. A stage that falls short will run again with new labels or a new cutoff. Once it meets the target accuracy, we will fill the remaining gaps manually and merge the automatic and manual results, dropping entities that do not exist, before the next stage begins. Repository resolution and model-assisted license steps will use the same gate on their own samples, as Section VII describes.

Extracting every tool manually, or checking every model output, would take about the same effort, and either path would limit how many skills, tools, and repositories we can cover. We will therefore accept a bounded residual error under the sample gate so the study can scale, in the same way a survey estimates from a sample instead of visiting every case. In extraction, the gate is meant to catch a model that marks the hosting folder, the publishing repository, an attribution link, the skill filename, or GitHub inside an address as a taught tool.

Future Work (Section VII)[8]

Repositories, licenses, and freedom groups (Section VII-D)

Added after the sentence that groups confirmed licenses by freedom

We will search published sources for a freedom category mapping and fix that mapping before any license is placed in a group.

[8] AP-11 Write the freedom mapping before seeing the data

4 Action points

These entries restate the raw notes in `raw/f1.md`[9] and `raw/f2.md`[10]. A recommendation stays a recommendation, an observation stays an observation, and a reviewer's uncertainty stays uncertainty. Each decision records whether the current paper already covers the point. Adopted means the draft changes in the way the reasoning describes. Skipped means the next step waits on a discussion that is still underway. Any comment that is not about `docs/paper/`, `scripts/`, or `docs/checkpoint_2/` was skipped. Each note has a stable code, assigned in document order, and the code stays if the note later moves.

Research questions

Checkpoint section `sections/initial_research.tex`. *Paper section* II.

AP-01 Counting unit

Point The counting unit needs a sentence in the paper. Once per distinct skill text is a defensible choice, but a skill copied into 5,000 repositories teaches its tool to 5,000 agents, and a template that stamps the same catalog everywhere will either dominate the ranking or disappear from it, depending on the unit. Report per distinct text and per repository next to each other. If the rankings differ, that is worth a paragraph on its own.

Source f1

Decision Not adopted

Reasoning RQ1 asks which tools appear most often in skill files. RQ2 asks how the projects that publish those tools are licensed, once each license is grouped by freedom. Neither question asks for a second count per hosting repository or for two rankings set side by side. The paper already counts each tool once per distinct skill text, and the unit for RQ2 is the publishing project.

Related work

Paper section Research Questions and Related Work.

AP-02 Closer related work

Point Closer related work: Hong et al. (arXiv:2607.01456) and Zhang et al. (arXiv:2608.08453) on what `SKILL.md` files contain; Jia et al. (arXiv:2607.01136), who trace what skills depend on and call, which is next door to the extraction step; and the license-mining literature (Vendome et al. on license evolution, for one) for RQ2.

Source f1

Decision Skipped

[9] `https://github.com/nichmorgan/msr-data-challenge-2027/blob/main/docs/checkpoint_2/feedbacks/raw/f1.md`

[10] `https://github.com/nichmorgan/msr-data-challenge-2027/blob/main/docs/checkpoint_2/feedbacks/raw/f2.md`

Reasoning The authors need to review Hong et al., Zhang et al., Jia et al., and Vendome et al. before adding them. We thank the reviewer for the suggestion, and we will evaluate it for the next versions.

Method

Checkpoint section `sections/technical_details.tex`. *Paper section* Method.

AP-03 Seeded skill-body sample

Point The pilot has not touched the input the study will measure. Both model runs read companion text, and the twelve-skill trial was read by eye. Draw a seeded sample of skill bodies using the population rules, run the label-only model, and hand-label 50 of the hits. A rough precision estimate on that input would do it.

Source f1

Decision Not adopted

Reasoning Future Work already labels the validation sample and records precision, recall, and agreement. Limitations says that check has not run. f2 asks for the same pilot on skill bodies. The draft keeps that check as future work.

AP-04 Chunk skill bodies before labeling

Point The 384-character clip may hit the 384-token limit on the full run. A skill body averages several thousand characters and names tools throughout, so without chunking the extractor reads only the first paragraph and misses most of what it is meant to count. Maybe chunk the body, run each chunk, and merge hits per skill. That is a few lines of code, but it needs to be in place before the validation sample is labeled, or the recall measured will not be the recall obtained.

Source f1

Decision Skipped

Reasoning Chunking is required if a model reads the skill body. The authors are discussing labeling by hand instead of using a model.[11] If the work continues with a model, chunking will be a must have.

AP-05 Cutoff, sample size, and agreement subset

Point A score cutoff of 0.8 and a precision of 80% are different metrics, and nothing ties them together. After the validation sample is labeled, sweep the cutoff, plot precision and recall against it, and pick the cutoff from the plot. State the sample size now, before labeling. Do the same for the agreement subset.

Source f1

Decision Not adopted

[11] Section 2

Reasoning The sweep, the precision and recall plot, and the sample sizes are the manual-validation stage. Future Work already labels that sample and records precision, recall, agreement, and the errors. Method already reruns a stage with a new cutoff when it falls short. Limitations says those rates are unreported because the sample has not been labeled, and the appendix leaves the sample size undecided.

AP-06 Sample rows in the notebook

Point In `explore_entities.ipynb`, the 500-row draw samples each column separately with the same seed. It happens to line up, but sample rows instead.

Source f1

Decision Not adopted

Reasoning The 500-row draw is in `explore_relations.ipynb`, not `explore_entities.ipynb`. It is `select(pl.all().sample(n=500,seed=SEED))`. That samples each column on its own, and the same seed draws the same positions. On Polars 2.1.0 the draw matches `DataFrame.sample` with that seed, including order. Dropping the seed crosses the columns, and `set_random_seed` does not stop that. This Polars version has no `LazyFrame.sample`, so the expression is how the lazy scan samples rows. No notebook change.

AP-07 Shorter method sentences

Point Some sentences in the method run long enough that they had to be re-read multiple times. Shorter ones would help MSR reviewers.

Source f1

Decision Adopted

Reasoning We will rewrite Method with shorter and clearer sentences.

RQ2 resolution

Paper sections Method and Future Work.

AP-08 Resolving every name may stall the plan

Point RQ2, as written, is where the plan could stall. Resolving every tool name to a publishing repository using a reasoning model plus web search is slow, hard to rerun (the rerun problem is already noted in the work), and the long tail of names is probably in the thousands.

Source f1

Decision Not adopted

Reasoning Future Work already resolves each confirmed tool one at a time and records that an external service can answer differently on a rerun. The draft keeps that stage as written.

AP-09 Resolve the top N tools

Point Resolve the top N tools by skill count and report what share of all mentions they cover.

Source f1

Decision Not adopted

Reasoning Future Work resolves each confirmed tool and answers the second question with the share of publishing projects in each freedom group. A top-N resolution and the share of mentions those tools cover are part of that unrun stage. The draft leaves them there.

AP-10 Deterministic license sources first

Point Use deterministic sources before any model. PyPI and npm return a license field through their APIs, and GitHub's license endpoint returns an SPDX ID.

Source f1

Decision Not adopted

Reasoning A PyPI, npm, or GitHub license lookup returns a stable field only after the package or repository is already known. Entity extraction is the uncertain step, and the name it produces is not a registry identifier. The authors are considering labeling tools by hand because of that.[12] Those APIs would leave the search tied to a generated query, so they stay out of the plan.

AP-11 Write the freedom mapping before seeing the data

Point Write the license-to-freedom mapping down before seeing the data, using the OSI or FSF definitions it follows.

Source f1

Decision Adopted

Reasoning We will search published sources for a freedom category mapping, and we will fix that mapping before any license is placed in a group.

Set aside

These points are not changes to the paper.

- **AP-12.** Dates on the remaining-work list belong to the checkpoint report.
- **AP-13.** The report length belongs to the checkpoint page limit.
- **AP-14.** The README note is about the README still listing three research questions.

[12] Section 2